KUBERNETES PREPARATION · MY LEARNING NOTES · SESSION 08

Secrets and ConfigMaps

Your notes, cleaned up from dictation + screenshots

Quick recall

- **Secret** — for anything *sensitive* (passwords, API keys, SSH keys) that must never sit in plain text inside a YAML that goes into Git. **Encoded (base64), not encrypted.**
- **ConfigMap** — for changing an already-built (read-only) image's runtime *behavior/config* without rebuilding it — e.g. making the same image listen on a different port per environment.
- Both are consumed by a Pod the same two ways: as an **environment variable**, or as a **mounted file/volume**.

PART 1 · SECRETS

What is a Secret, and why it exists

Anything sensitive — usernames/passwords, API keys, SSH keys — cannot be written in plain text inside a YAML file. In production, every YAML eventually gets pushed into a VCS (Git). If a password sits in that YAML as plain text, **everyone with repo access can read it** the moment it lands in Git — a real security compromise. Kubernetes Secrets exist to avoid exactly this.

```
YAML (user: alok, pass: xxxx, in plain text)  --push-->  Git / VCS
                    ^
                    |  THIS is the problem a Secret solves
```

⚠ Encoded ≠ Encrypted (the classic gotcha)

- **Encoding** — reversible. Anything encoded can be decoded straight back to its original value.
- **Encryption** — not reversible without a key. Cannot be undone by just "decrypting" it back.
- **Kubernetes Secrets are only base64-*encoded***, not encrypted. Base64 is trivially reversible by anyone with the value — a Secret's real protection is Kubernetes RBAC (who's *allowed* to `get` / `describe` it), not the encoding itself.
- Structurally, a Secret is just **key = value** pairs, same shape as labels.

Creating a Secret — a 2-step process

1. Create the Secret.
2. Inject it into a Pod.

```
# Step 1: create a secret named "myuser" with two literal key/value pairs
kubectl create secret generic myuser --from-literal username=alok --from-literal passwor
# secret/myuser created

# List secrets
kubectl get secrets
# NAME     TYPE     DATA   AGE
# myuser   Opaque   2      9s
```

Inspecting a Secret — `describe` hides values, `-o yaml` shows the (encoded) values

```
kubectl describe secret myuser
# Name:          myuser
# Namespace:     default
# Labels:        <none>
# Annotations:   <none>
# Type:          Opaque
# Data
# ====
# password:  6 bytes
# username:  4 bytes
```

`describe` deliberately only shows byte counts, never the actual value. To see the (still-encoded) value:

```
kubectl get secrets myuser -o yaml
# apiVersion: v1
# data:
#   password: c3VuZGF5
#   username: YWxvaw==
# kind: Secret
# metadata:
#   name: myuser
#   namespace: default
# type: Opaque
```

Decoding a Secret value

```
echo c3VuZGF5 | base64 --decode
# sunday
echo YWxvaw== | base64 -d
# alok
```

Anyone who can run `kubectl get secret -o yaml` against your Secret can trivially recover the original plaintext this way — reinforcing that base64 is **not** a security control by itself.

Using a Secret in a Pod — as environment variables

```
apiVersion: v1
kind: Pod
metadata:
  name: secret
spec:
  containers:
    - name: web
      image: lovelearnlinux/webserver:v1
      env:
        - name: SECRET_USERNAME
          valueFrom:
            secretKeyRef:
              name: myuser      # the Secret to pull from
              key: username     # the specific key inside that Secret
        - name: SECRET_PASSWORD
          valueFrom:
            secretKeyRef:
              name: myuser
              key: password
```

The env var name (`SECRET_USERNAME` / `SECRET_PASSWORD`) is arbitrary — pick whatever name the application expects (e.g. a MySQL image's own expected env var names).

Verifying it actually landed in the Pod:

```
kubectl describe pod secret
# Environment:
#   SECRET_USERNAME:  <set to the key 'username' in secret 'myuser'>  Optional: false
#   SECRET_PASSWORD:  <set to the key 'password' in secret 'myuser'>  Optional: false

kubectl exec -it secret -- /bin/bash
[root@secret /]# env
# LANG=en_US.UTF-8
# SECRET_PASSWORD=sunday
# HOSTNAME=secret
# SECRET_USERNAME=alok
#  ...
```

Only someone with permission to `exec` into the Pod (or read the Secret directly) can ever see the real values — that's the actual access boundary, not the base64 encoding.

PART 2 · CONFIGMAP

The problem ConfigMap solves: images are read-only once built

Once a Docker image is built, **it is read-only** — you cannot reach into a running container and permanently change what's baked into that image. But you often need to change a container's *runtime behavior* after the image already exists, without rebuilding it. That's what a ConfigMap is for: inject/override configuration into an already-built image's container at start time.

Worked example — one image, two environments, two ports

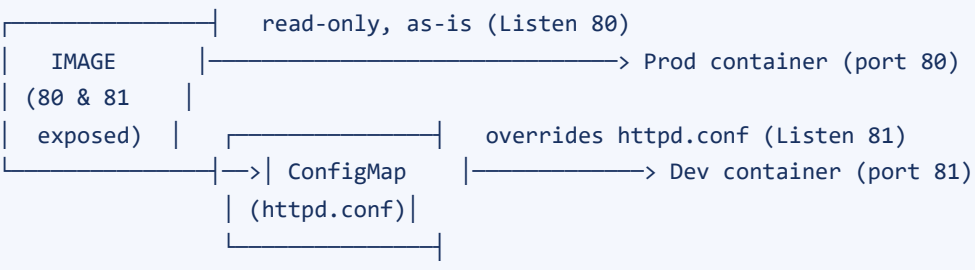
Developer's Dockerfile:

```
FROM centos:7
RUN yum install httpd -y
COPY index.html /var/www/html/
EXPOSE 80
```

Installing `httpd` via `yum` creates a default config file at `/etc/httpd/conf/httpd.conf`, which by default contains `Listen 80` — which is *why* the Dockerfile exposes port 80: it matches httpd's own default.

> **The conflict:** Prod wants this app on port 80 — fine, the image already does that. But **Dev wants the same image running on port 81.** You can't just add `EXPOSE 81` to the Dockerfile, because the actual `httpd.conf` baked into the image still says `Listen 80` — exposing a port in Docker doesn't change what the application inside is actually listening on. Rebuilding a Dev-specific image just to flip one config line defeats the point of a single shared image.

The fix: build the image once with **both** ports exposed (`EXPOSE 80 81`), keep the baked-in default at `Listen 80` for Prod, and for Dev, mount a **ConfigMap** containing a modified `httpd.conf` (`Listen 81`) over the container's config path at startup — same image, different effective config per environment.



Step 1 — Prod: run the image as-is (port 80)

```
# prod-web.yaml
apiVersion: v1
kind: Pod
metadata:
  name: prodweb
spec:
  containers:
    - name: web
      image: lovelearnlinux/customweb:v1
```

```
kubectl create -f prod-web.yaml
# pod/prodweb created

kubectl get pods -o wide
# NAME      READY   STATUS    IP             NODE
# prodweb   1/1     Running   172.16.10.80   nodeone

curl http://172.16.10.80
# this is a custom webserver
# which can run on port 80 and 81
```

Works immediately — the image's baked-in `httpd.conf` already listens on 80.

Step 2 — Dev: build a modified `httpd.conf` and turn it into a ConfigMap

Take a local copy of the container's `httpd.conf` and change the `Listen` line:

```
ServerRoot "/etc/httpd"
...
#Listen 12.34.56.78:80
Listen 81
```

Turn that file into a ConfigMap:

```
kubectl create configmap devweb --from-file=httpd.conf
# configmap/devweb created

kubectl get configmap
# NAME              DATA   AGE
# devweb            1      5s
# kube-root-ca.crt  1      9d

kubectl describe configmap devweb
```

Step 3 — Dev Pod: mount the ConfigMap over the config directory

```
# dev-web.yaml
apiVersion: v1
kind: Pod
metadata:
  name: devweb
spec:
  containers:
    - name: mypod
      image: lovelearnlinux/customweb:v1
      volumeMounts:
        - name: webroot
          mountPath: "/etc/httpd/conf/"
          readOnly: true
  volumes:
    - name: webroot
      configMap:
        name: devweb
```

```
kubectl create -f dev-web.yaml
# pod/devweb created

kubectl get pods
# NAME      READY   STATUS    AGE
# devweb    1/1     Running   6s
# prodweb   1/1     Running   8m31s
# secret    1/1     Running   26m
```

Proving it — same image, two different effective ports

```
kubectl get pods -o wide
# NAME      IP             NODE
# devweb    172.16.10.81   nodeone
# prodweb   172.16.10.80   nodeone
# secret    172.16.10.82   nodeone

curl http://172.16.10.80
# this is a custom webserver
# which can run on port 80 and 81      ← prodweb, default config, port 80 works

curl http://172.16.10.81
# curl: (7) Failed to connect to 172.16.10.81 port 80: Connection refused   ← devweb no

curl http://172.16.10.81:81
# this is a custom webserver
# which can run on port 80 and 81      ← devweb, ConfigMap-overridden config, now on 81
```

Confirmed from inside the Pod itself:

```
kubectl exec -it devweb -- /bin/bash
[root@devweb /]# cat /etc/httpd/conf/httpd.conf | less
# ...
# #Listen 12.34.56.78:80
# Listen 81
```

Same image (`lovelearnlinux/customweb:v1`) in both Pods — the only difference is the ConfigMap mounted over `/etc/httpd/conf/` in `devweb`. This is the entire point of a ConfigMap: change a read-only image's runtime behavior per environment, without ever rebuilding it.

Practice Tasks

☐ Create a Secret from two `--from-literal` values, then try `kubectl describe secret` (confirm it only shows byte counts) vs. `kubectl get secret -o yaml` (confirm it shows base64) — internalize the difference before decoding anything.

☐ Decode a Secret value with `base64 --decode` and reason out loud why this proves encoding is not encryption.

☐ Inject a Secret into a Pod as two environment variables via `secretKeyRef`, then `kubectl exec` in and confirm both values with `env`.

☐ Build the Dockerfile from this session (`httpd` on CentOS 7, `EXPOSE 80 81`), run it once unmodified (Prod-style) and confirm `Listen 80` is the default straight out of the image.

☐ Create a ConfigMap from a locally modified `httpd.conf` (`Listen 81`), mount it into a second Pod using the same image, and confirm via `curl` that the two Pods now genuinely listen on different ports despite sharing one image.

☐ Deliberately reference a Secret key that doesn't exist in a Pod's `secretKeyRef` and observe the resulting Pod error state (`CreateContainerConfigError`) via `kubectl describe pod`.

What I practiced

- Created a Secret (`myuser`) from two literals (username/password), inspected it with `describe` (hidden values) vs. `-o yaml` (base64 values), and decoded both back to plaintext with `base64 --decode`.
- Injected that Secret into a Pod as two environment variables via `secretKeyRef`, verified the binding with `kubectl describe pod`, then confirmed the real values by `exec`-ing in and running `env`.
- Built the Dockerfile → image → Prod/Dev port-conflict scenario end to end: ran the image as-is for Prod (port 80), built a ConfigMap from a modified `httpd.conf` (`Listen 81`) for Dev, mounted it over `/etc/httpd/conf/` read-only, and proved via `curl` (80 works on Prod's IP, fails on Dev's IP, succeeds on Dev's IP at `:81`) and a direct `cat` inside the Dev Pod that the same image now runs two different effective configs per environment.

Merged into the master reference notes: k8s-master-notes.md §5 (Configuration & Secrets) — added a "Secrets & ConfigMaps — Deep Dive" addendum with the encoding-vs-encryption distinction and the read-only-image ConfigMap override pattern.